1. Exercise 1

1.1. Requirements

Goal: Implement a vehicle position control task, and test that the vehicle reaches the required position. Add the proper Jacobian variable, its activation function, and the desired reference rate.

To test that the task is working properly

- Initialize the vehicle far away from the seafloor. An example position could be:
  eta = [10.5 35.5 −36 0 0 pi/2] (x,y,z,roll,pitch,yaw)
- Give a target position that is also sufficiently away from the seafloor, e.g.,
  w_vehicle_goal_position = [10.5 37.5 −38]

1.2. Solution

- `TaskVehiclePosition.m`:
  - The `updateReference` function computes the desired vehicle velocity based exclusively on the linear position error, as the exercise does not require orientation control.
  - The `updateJacobian` function maps the vehicle's linear velocities to the world frame using `robot.wTv(1:3, 1:3)`. The first seven columns (arm joints) and the last three columns (vehicle orientation) are set to zero because those degrees of freedom are ignored by this task.
  - The `updateActivation` function enables control for these three linear axes by returning a 3x3 identity matrix.
- `RobustMain.m`
  - The custom task is instantiated and prepared for the controller:

    ```
    task_vehicle = TaskVehiclePosition();
    ```

  - The action is then registered and selected as the active priority stack:

    ```
    actionManager.addAction("move_vehicle", move_vehicle_tasks);
    actionManager.setCurrentAction("move_vehicle");
    ```

  - Finally, the task stack is assigned to the logger for runtime monitoring:

    ```
        task_set = move_vehicle_tasks;
    ```

2. Exercise 2

2.1. Requirements

Goal: Implement a horizontal attitude control task, and test that the vehicle reaches the required orientation. Add the proper Jacobian variable, its activation function, and the desired reference rate.

To test that the task is working properly

- Initialize the vehicle far away from the seafloor. An example position could be:
  eta = [10.5 35.5 −36 -pi/3 pi/3 pi/2] (x,y,z,roll,pitch,yaw)
- Require that the norm of the misalignment between the vehicle's z vector and the absolute z-axis is less than 0.2

2.2. Solution

- `TaskVehicleMisalignment.m`:
  - The `updateReference` function computes the desired angular velocity to correct the misalignment, obtained through the cross product between the vehicle's z-axis and the absolute z-axis.
  - The `updateJacobian` function places the rotation matrix `wTv(1:3, 1:3)` in the last three columns of the Jacobian. This maps the correction exclusively to the vehicle's angular velocities, ignoring the arm joints (first 7 columns) and the linear velocities (columns 8-10).
  - The `updateActivation` function handles the inequality: it returns `obj.A = a * eye(3)`, where `a` is set to 1 to fully activate the correction if the error exceeds the 0.2 threshold, and 0 to disable the task when the attitude is within safety limits.
- `RobustMain.m`:
  - The custom task is instantiated:

    ```
    task_misalignment = TaskVehicleMisalignment();
    ```

  - The task is inserted into the priority stack and the action is registered, ensuring `task_misalignment` is assigned an appropriate priority relative to the other tasks (e.g., higher priority than the position task):

    ```
    vehicle_tasks = {task_position, task_misalignment}; % task priorities
    actionManager = ActionManager(robotModel);
    actionManager.addAction("move_vehicle", vehicle_tasks);
    actionManager.setCurrentAction("move_vehicle");
    ```